

# PLACEMENT PORTAL APPLICATION

## Final Project Report

By SHIKHAR SINGH

23f2003771 | Modern Application Development - 2 | January, 2026 Term

### AUTHOR

**Name** - Shikhar Singh

**Roll No.** - 23F2003771

**Email** - [23f2003771@ds.study.iitm.ac.in](mailto:23f2003771@ds.study.iitm.ac.in)

**About Me** - I use data to build cool and innovative applications that solve real-world problems. I studied Math and Computer Applications, and I really love to code. I'm good at Python, making websites, and machine learning. When I'm not coding, I like to play games and read books.

## Project Details

**Project Title:** Placement Portal Application

**Problem Statement:** To build a web-based portal that automates campus recruitment by allowing Companies and Students to register (while Admin has login-only access), and manage job drives, and application tracking in one place, replacing manual spreadsheets.

**Approach:** I developed the application using a decoupled architecture with a Flask RESTful API for the backend and VueJS for the frontend. The system uses role-based access control (JWT Tokens), SQLite for data storage, and Celery with Redis for handling background tasks such as CSV exports and email reminders.

## AI/LLM Declaration

AI tools (ChatGPT, Gemini) were used in a limited capacity around 8–12% mainly for initial project bootstrapping, generating sample model structures, and improving documentation formatting.

The overall system design, database schema, core implementation logic, debugging, and full integration of the project were developed and completed independently by me.

## Technologies and Frameworks Used

- **Flask:** Used as the core backend framework to build the RESTful API.
- **VueJS:** Used to create a responsive and dynamic single-page user interface.
- **SQLite:** Used as the relational database to store user and drive information.
- **Redis:** Used as the message broker for Celery and for caching application data.
- **Celery:** Used to handle background batch jobs like monthly reports and CSV exports.
- **Bootstrap:** Used for styling the UI components and ensuring a clean layout.
- **Flask-SQLAlchemy:** Used as the ORM to interact with the SQLite database programmatically.
- **JWT:** Used for secure authentication and authorization by generating tokens for user login sessions.
- **CORS:** Used to enable secure cross-origin requests between the frontend and backend APIs.

## Database Schema

The database consists of several tables including Users, Student Profiles, Company Profiles, Placement Drives, and Applications. It uses foreign keys to link students and companies to their respective drives and application statuses.

**User** — stores account details of all platform users  
(id, email, password\_hash, role, is\_active, created\_at)

**StudentProfile** — stores additional details of students (id, user\_id, full\_name, branch, year, cgpa, phone, resume\_path, is\_blacklisted)

**CompanyProfile** — stores details of companies participating in placements  
(id, user\_id, company\_name, hr\_contact, website, description, approval\_status, is\_blacklisted)

**PlacementDrive** — stores information about job/placement drives posted by companies  
(id, company\_id, drive\_name, job\_title, job\_description, salary, location, eligibility\_criteria, application\_deadline, interview\_type, status, created\_at)

**Application** — stores records of student applications to placement drives  
(id, student\_id, drive\_id, applied\_at, status, remark)

### Relationships

One-to-One → **User** → **StudentProfile**

One-to-One → **User** → **CompanyProfile**

One-to-Many → **CompanyProfile** → **PlacementDrive**

One-to-Many → **StudentProfile** → **Application**

One-to-Many → **PlacementDrive** → **Application**

## ER Diagram



## API Resource Endpoints

The following RESTful resources were implemented using flask-restful:

- **UserRegistration** ('/'): Handles new user signups (POST).
- **UserLogin** ('/login'): Authenticates users and starts sessions (POST).
- **AdminDashboard** ('/admin/dashboard', '/admin/dashboard/<int:id>'): Allows Admin to view stats and manage users, company applications and drives (GET, PUT).

- **CompanyDashboard** ( `' /company/dashboard '` , `' /company/dashboard/<int:id> '` ): Allows companies to post drives, view applicants, and change students' application status (GET, POST, PUT).
- **StudentDashboard** ( `' /student/dashboard '` , `' /student/dashboard/<int:id> '` ): Allows Students to view drives and apply for open positions (GET, POST).
- **CSVExport** ( `' /export/csv '` ): Triggers the background job to export student or company data (POST).
- **Logout** ( `' /logout '` ): Clears user sessions (POST).

## Implemented Features

- **Role-Based Access:** Distinct functionalities for Admin, Company, and Student roles.
- **Drive Management:** Companies can create placement drives; Admin can approve or reject them.
- **Application Tracking:** Students can apply for ongoing drives and track their status (Applied/Selected/Shortlisted/Waitlisted/Rejected).
- **Scheduled Jobs:**
  - **Daily Reminders** – Automated reminders for upcoming interviews via email.
  - **Monthly Reports** – Admin & Companies receive an HTML activity report via email on the 1st of every month.
- **Async Batch Jobs:** Students and Companies can trigger a "Export to CSV" task that runs in the background and sends emails with dashboard summary.
- **Caching:** Implemented Redis caching on frequently accessed data like Dashboards for admin and users to improve performance.

## Video Link

Video Demonstration of my project is available here:

<https://drive.google.com/file/d/1n8T8IppqOyo3FNx5pRJ95B73qVUqXea2y/view>